

DSA & Advanced DSA Roadmap

Phase 11 — Trees

Learning Objectives

By the end of this phase, learners will be able to:

- Implement a binary tree and a binary search tree (BST) from scratch, including insertion and search.
- Implement all four traversal orders — preorder, inorder, postorder, level order — both recursively and iteratively.
- Apply DFS and BFS to trees to compute properties (height, diameter, path sums) and to construct or validate tree structures.
- Find the lowest common ancestor of two nodes, distinguishing the BST-specific approach from the general binary tree approach.
- Solve Easy to Medium level tree problems independently, and approach Hard tree problems (e.g. Binary Tree Maximum Path Sum) by extending the 'compute bottom-up, track a global best' technique from Diameter of Binary Tree.

Implementation & Internals

Core Concepts

- Binary Trees — each node has at most two children (left, right); no ordering property is assumed.
- BST (Binary Search Tree) — a binary tree with the added invariant that every node's left subtree contains only smaller values and its right subtree only larger values, enabling $O(\log n)$ search on a balanced tree.
- Recursive Traversals — preorder (root, left, right), inorder (left, root, right), postorder (left, right, root); each is a 3-line recursive function once the pattern is internalized.
- Iterative Traversals — reproducing each recursive traversal using an explicit stack (DFS orders) or queue (level order, first seen in Phase 5), useful when recursion depth is a concern.

Implementation Exercises

- Build a binary search tree from scratch supporting insert and search, then verify correctness by checking an inorder traversal produces sorted output.
- Implement preorder, inorder, and postorder traversal both recursively and iteratively (using an explicit stack), and confirm both versions produce identical output on the same tree.
- Implement Validate Binary Search Tree using the bounds-propagation technique (each recursive call carries a valid (min, max) range) rather than the simpler but less general 'check inorder is sorted' approach.

Pattern Mastery

Pattern 1: DFS

Concepts

- Depth-first traversal of a tree using recursion (implicit stack) or an explicit stack.
- Choosing which DFS order (preorder, inorder, postorder) fits a given problem — inorder for BST-ordered output, postorder for bottom-up aggregation.

Practice Problems

- Return the inorder traversal of a binary tree's node values.

- Return the preorder traversal of a binary tree's node values.
- Given the root of a binary tree, determine whether it is a valid binary search tree.

LeetCode

- [Binary Tree Inorder Traversal — leetcode.com/problems/binary-tree-inorder-traversal](https://leetcode.com/problems/binary-tree-inorder-traversal)
- [Binary Tree Preorder Traversal — leetcode.com/problems/binary-tree-preorder-traversal](https://leetcode.com/problems/binary-tree-preorder-traversal)
- [Validate Binary Search Tree — leetcode.com/problems/validate-binary-search-tree](https://leetcode.com/problems/validate-binary-search-tree)

Pattern 2: BFS

Concepts

- Level-by-level traversal of a tree using a queue, first introduced in Phase 5 with Binary Tree Level Order Traversal.
- Using BFS to compare structure across a tree symmetrically (left subtree vs. right subtree) rather than only computing per-level output.

Practice Problems

- Given the root of a binary tree, check whether it is a mirror of itself (symmetric around its center).

LeetCode

- [Symmetric Tree — leetcode.com/problems/symmetric-tree](https://leetcode.com/problems/symmetric-tree)

Note: Binary Tree Level Order Traversal (the canonical BFS-on-trees problem) was already assigned in Phase 5 as the BFS Foundation exercise and is not repeated here.

Pattern 3: Tree Height

Concepts

- Computing a node's height as $1 + \max(\text{height of left child}, \text{height of right child})$, with height 0 for a null node.
- Recognizing that 'depth' (introduced in Phase 9 via Maximum Depth of Binary Tree) and 'height' are the same computation viewed from different reference points.

Practice Problems

- Given the root of a binary tree, check whether two given trees are structurally identical with the same node values.

LeetCode

- [Same Tree — leetcode.com/problems/same-tree](https://leetcode.com/problems/same-tree)

Note: Maximum Depth of Binary Tree (the canonical height/depth problem) was already assigned in Phase 9 as a tree-recursion preview and is not repeated here; Same Tree is used instead as this phase's height-comparison exercise.

Pattern 4: Diameter

Concepts

- Computing, for every node, the longest path passing through it as the sum of its left and right subtree heights, then tracking the maximum across all nodes in a single bottom-up pass.
- Recognizing the 'return one value to the parent, but track a separate global answer' technique — the height is returned upward, but the diameter is recorded in a variable outside the recursive return value.

Practice Problems

- Given the root of a binary tree, return the length of the diameter of the tree (the longest path between any two nodes, which may or may not pass through the root).

LeetCode

- [Diameter of Binary Tree — leetcode.com/problems/diameter-of-binary-tree](https://leetcode.com/problems/diameter-of-binary-tree)

Pattern 5: Path Sum

Concepts

- Tracking a running sum (or remaining target) as you descend from root to leaf, checking the condition only once a leaf is reached.
- Extending a single root-to-leaf existence check into collecting all qualifying paths, which requires tracking and backtracking the current path list (connecting back to Phase 10's backtracking template).

Practice Problems

- Given the root of a binary tree and a target sum, determine whether the tree has a root-to-leaf path such that the values along the path add up to the target.
- Given the root of a binary tree and a target sum, return all root-to-leaf paths where the values along the path sum to the target.

LeetCode

- [Path Sum — leetcode.com/problems/path-sum](https://leetcode.com/problems/path-sum)
- [Path Sum II — leetcode.com/problems/path-sum-ii](https://leetcode.com/problems/path-sum-ii)

Pattern 6: Lowest Common Ancestor

Concepts

- In a BST, using the ordering property to decide whether to descend left, descend right, or stop at the current node, without needing to search both subtrees.
- In a general binary tree (no ordering property), searching both subtrees and combining results: if both children report finding one of the two target nodes, the current node is the LCA.

Practice Problems

- Given a binary search tree and two nodes, find their lowest common ancestor using the BST ordering property.
- Given a general binary tree (no BST property) and two nodes, find their lowest common ancestor.

LeetCode

- [Lowest Common Ancestor of a Binary Search Tree — leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree](https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree)
- [Lowest Common Ancestor of a Binary Tree — leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree](https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree)

Pattern 7: Tree Construction

Concepts

- Rebuilding a unique binary tree from two traversal orders, using the property that a preorder traversal's first element is always the current subtree's root, and using that root's position in the inorder traversal to split the remaining nodes into left and right subtrees.
- Recognizing this as a divide-and-conquer application (Phase 9) specialized to trees: split, recurse on each half, combine by attaching as children.

Practice Problems

- Given a preorder traversal and an inorder traversal of the same binary tree (with unique values), reconstruct and return the original tree.

LeetCode

- [Construct Binary Tree from Preorder and Inorder Traversal — leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal](https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal)

Phase Assessment

Implementation Tasks

- Build a binary search tree from scratch with insert and search operations.
- Implement all three DFS traversal orders both recursively and iteratively, and the BFS level-order traversal using a queue.
- Analyze and explain why Validate Binary Search Tree requires passing a (min, max) bound down through recursive calls, rather than only comparing each node to its immediate parent.

Recommended LeetCode Target

Easy Problems

- leetcode.com/problems/binary-tree-inorder-traversal
- leetcode.com/problems/binary-tree-preorder-traversal
- leetcode.com/problems/same-tree
- leetcode.com/problems/symmetric-tree
- leetcode.com/problems/diameter-of-binary-tree
- leetcode.com/problems/path-sum

Medium Problems

DFS

- leetcode.com/problems/validate-binary-search-tree

Path Sum

- leetcode.com/problems/path-sum-ii

Lowest Common Ancestor

- leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree
- leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree

Tree Construction

- leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal

Hard Problems

- leetcode.com/problems/binary-tree-maximum-path-sum

On scope and dedup

This phase has 12 distinct problems — the largest set since Phase 1 — reflecting its 7 named patterns, the most of any phase so far. Binary Tree Maximum Path Sum is included as the Hard-tier capstone: it directly extends the Diameter of Binary Tree technique (bottom-up computation plus a tracked global best) to node values instead of edge counts. None of the 12 problems overlaps with any problem used in Phases 0–10, including the tree-recursion previews already placed in Phase 9 and the BFS-on-trees example already placed in Phase 5.

Coding Challenges (Assessment Day)

- Validate Binary Search Tree
- Diameter of Binary Tree

- Path Sum II
- Lowest Common Ancestor of a Binary Tree
- Construct Binary Tree from Preorder and Inorder Traversal
- Binary Tree Maximum Path Sum

Expected Outcome

Students should be able to:

- Implement a binary search tree from scratch and all four traversal orders, both recursively and iteratively.
- Recognize and apply the seven tree patterns covered: DFS, BFS, height, diameter, path sum, lowest common ancestor, and tree construction.
- Solve unseen tree problems using pattern-based thinking rather than memorized solutions.
- Complete Easy and Medium tree questions independently, and approach Hard tree problems by extending a known bottom-up technique to a new objective.